

Angular Foundations

Getting Started with Angular

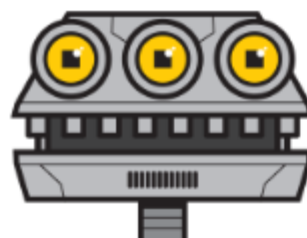


Jim Cooper

Software Engineer

@jimthecoop | jcoop.io



 JOE'S ROBOT SHOP	Catalog Cart	
	Large Cyclops A robot head with an unusually large eye and teloscpic neck -- excellent for exploring high spaces. Part Type: Heads	\$1,220.50 \$976.40 Buy
	Spring Base A spring base - great for reaching high places. Part Type: Bases	\$1,190.50 Buy
	Articulated Arm An articulated arm with a claw -- great for reaching around corners or working in tight spaces. Part Type: Arms	\$275.00 Buy
	Friendly Bot A friendly robot head with two eyes and a smile -- great for domestic use. Part Type: Heads	\$945.00 \$756.00 Buy
	Shredder A large three-eyed head with a shredder for a mouth -- great for crushing light medals or shredding documents.	\$1,275.50 Buy

Anatomy of a Component

Component Class

```
import { Component } from '@angular/core';
import { IProduct } from '../product.model';

@Component({...})
export class CatalogComponent {
  products: IProduct[];
  ...
}
```



Component Template

```
<ul>
  @for (product of products; let i = product.id) {
    <li class="product-item" (click)="addToCart()">
      {{ product.name }}
    </li>
  }
</ul>
```



Component Styles (CSS)

```
ul {
  margin: 0 100px;
  border-top: 2px solid #999;
}

.product {
  display: flex;
  justify-content: space-between;
  padding: 20px 25px;
  border-bottom: 2px solid #999;
}
```

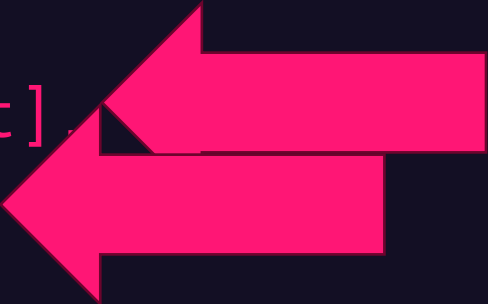


Anatomy of a Component

Component Class

```
import { Component } from '@angular/core';
import { IProduct } from './product.model';
import { ProductDataService } from './product-data.service';
import { ProductDetailsComponent } from './product-details.component';

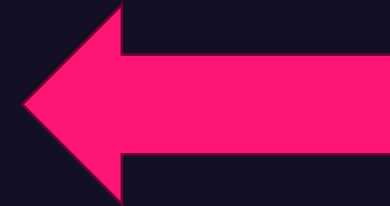
@Component({
  standalone: true,
  imports: [ProductDetailsComponent],
  providers: [ProductDataService],
})
export class CatalogComponent {
  products: IProduct[];
  ...
}
```



Dependency Injection

Component Class

```
...  
  
@Component({  
  standalone: true,  
  providers: [ProductDataService],  
})  
export class CatalogComponent {  
  products: IProduct[];  
  
  constructor(private productService: ProductDataService)  
    this.products = this.productService.getProducts();  
  }  
}
```



Routing

app.routes.ts

```
const routes: Routes = [  
  { path: '', component: HomeComponent },  
  { path: 'catalog', component: CatalogComponent },  
  { path: 'cart', component: CartComponent },  
  ...  
];
```



Development Environment Dependencies



<http://nodejs.org>



<http://code.visualstudio.com/download>



TypeScript Features

Static typing

Interfaces

Class properties

**Public/Private
accessibility**




```
let name : string;  
let age : number;  
let birthDate : date;  
age = "seven"; //error
```

Static Typing



```
interface ICat {  
  name: string;  
  age: number;  
}
```

TypeScript Interfaces



```
interface ICat {  
  name: string;  
→ age: number;  
}
```

```
let fluffy:ICat;
```

```
fluffy =  
  {  
    name: 'Fluffy',  
→   age: 'seven'  
  };
```

TypeScript Interfaces



```
interface ICat {  
  name: string;  
  age: number;  
}  
  
let fluffy:ICat  
    fluffy =  
    {  
      name: 'Fluffy'  
    };
```

TypeScript Interfaces



```
interface ICat {  
  name: string;  
  age?: number;  
}  
  
let fluffy:ICat  
  
fluffy =  
  {  
    name: 'Fluffy'  
  };
```

TypeScript Interfaces




```
class Cat {  
    constructor (name) {  
        this.name = name;  
    }  
}
```

TypeScript Class Properties



```
class Cat {  
    name: string;  
    constructor (name) {  
        this.name = name;  
    }  
}
```

TypeScript Class Properties



```
class Cat {  
  name: string;  
  color: string;  
  constructor (name) {  
    this.name = name;  
  }  
}
```

TypeScript Class Properties




```
class Cat {  
→ name: string;  
  constructor (name) {  
    this.name = name;  
  }  
→ speak() { console.log('meow'); }  
}
```

Public and Private Accessibility



```
class Cat {  
  name:string  
  constructor (name) {  
    this.name = name;  
  }  
  speak() { console.log('My name is: ' + this.name) }  
}
```

Public and Private Accessibility



```
class Cat {  
  name:string;  
  speak() { console.log('My name is: ' + this.name); }  
}
```

```
let fluffy = new Cat();  
console.log(fluffy.name);  
fluffy.speak();
```

Public and Private Accessibility



```
class Cat {  
  private name:string;  
  private speak() { console.log('My name is: ' + this.name); }  
}
```

```
let fluffy = new Cat();  
console.log(fluffy.name);  
fluffy.speak();
```

Public and Private Accessibility



```
class Cat {  
  private name:string;  
  private speak() { console.log('My name is: ' + this.name); }  
}
```

```
let fluffy = new Cat();  
console.log(fluffy.name); //compile-time error  
fluffy.speak(); // compile-time error
```

Public and Private Accessibility




```
class Cat {  
  private name: string;  
  private color: string;  
  constructor(name, color) {  
    this.name = name;  
    this.color = color;  
  }  
}
```

```
class Cat {  
  constructor(private name, private color) {  
  }  
}  
  
let fluffy = new Cat('Fluffy', 'White');
```

Public and Private Accessibility

